

One Person Lab

OPL Cloud 白皮书

让复杂知识工作跨越本机、私有数据与远端资源，仍然保持连续、受控、可复查

云端不应该迫使用户离开成果、转而管理基础设施。OPL Cloud 的设计目标，是让工作从本机自然延伸到在线 Workspace、私有数据、高性能计算和团队协作，同时把权限、成本、证据与专业判断放回各自正确的责任边界。

让复杂知识工作跨越本机、私有数据与远端资源，仍然保持连续、受控、可复查

2026-07-13

Public whitepaper

目录

1 定位摘要 2

2 为什么复杂知识工作需要不同的云 2

3 OPL Cloud 的答案：连续工作面，分层责任 2

4 OPL Cloud 的五大设计原则 3

4.1 一、成果优先于基础设施 3

4.2 二、本机与云端是同一条工作链 3

4.3 三、先计划，再授予资源权力 3

4.4 四、管理面不接管工作面 3

4.5 五、证据可以连接，不能越权裁决 4

5 OPL Cloud 的能力版图 4

6 标准任务生命周期 4

6.1 1. 从本机形成问题 4

6.2 2. 选择已验证的专业能力 4

6.3 3. 形成资源计划 4

6.4 4. 在关键边界批准 4

6.5 5. 在数据所在处执行 5

6.6 6. 把结果带回工作台 5

6.7 7. 交给领域独立审阅 5

6.8 8. 留下可继续的回执 5

7 三类用户路径 5

8 系统模型 5

9 专业工作流 6

10 五类信任机制 6

10.1 范围可信 6

10.2 执行可信 6

10.3 版本可信 6

10.4 结果可信 6

10.5 接力可信 6

11 本文边界 6

12 结语 7

发布日期：2026-07-13

适用对象：正在使用 AI 推进科研、基金、汇报、书籍或其他长期知识项目，希望在不改变工作方式的前提下使用在线工作台、团队资源与远端计算的个人、实验室和机构。

核心判断：云端不应该迫使用户离开成果、转而管理基础设施。OPL Cloud 的设计目标，是让工作从本机自然延伸到在线 Workspace、私有数据、高性能计算和团队协作，同时把权限、成本、证据与专业判断放回各自正确的责任边界。

1 定位摘要

OPL Cloud 不是一份云服务菜单，也不是把模型、容器、存储和后台拼在一起。它是 One Person Lab 的云端产品架构：用户继续围绕问题、材料、阶段、产物和审阅推进工作；系统在背后连接 AI、在线工作空间、组织策略、远端资源和证据回执。

本仓库承担 **OPL Cloud 目标产品架构与实现族导航**。本文解释为什么采用这样的设计、各部分如何协作，以及这种协作怎样改善用户体验；它不表示文中每项服务都已经交付。具体能力是否可用、运行到什么程度、如何计费 and 何时发布，以对应实现仓、机器合同、运行读回与正式公告为准。

OPL Cloud 希望兑现四个用户结果：

- 从本机开始，需要时再接入在线工作台或远端资源，不必重建项目。
- 私有数据可以留在机构边界内，计算去到数据所在的位置。
- 团队在关键动作前完成授权，日常工作不被管理后台打断。
- 结果回到原工作链，并带有足够的来源、环境、审阅和继续线索。

2 为什么复杂知识工作需要不同的云

一次普通云任务通常可以用“上传、运行、下载”描述。长期知识工作并不如此简单。

研究者可能先在本机整理问题和材料，随后发现分析需要医院内网数据、实验室高性能计算集群和特定软件环境。团队需要知道谁可以访问数据、任务会消耗多少资源、输出应回到哪里；领域负责人还要判断结果是否足以支持论文中的结论。数周后，另一位成员需要理解这次分析从哪里来，并接着修改。

如果每遇到一个问题就切换到一套独立系统，用户会同时维护本地文件、云盘、作业队列、模型账号、审批消息、环境说明和人工日志。真正困难的不是缺少工具，而是这些工具没有围绕同一个成果协作。

OPL Cloud 因此不从“云上能提供多少资源”出发，而从四组矛盾出发：

用户需要	常见代价	OPL Cloud 的设计选择
使用更强的 AI 与计算	被迫离开当前工作台	本机 App 与在线 Workspace 使用连续的工作模型
使用私有数据与机构资源	复制数据或暴露凭据	让 Fabric 把执行绑定到资源，而不是集中搬运所有数据
团队治理权限与成本	每一步都经过管理后台	Console 只在组织策略和批准边界介入
长期解释和复查结果	依赖成员记忆与手工记录	Ledger 保存引用与回执，专业结论仍由领域 owner 判断

这四个选择共同指向一个原则：**让工作保持连续，让权力保持分离。**

3 OPL Cloud 的答案：连续工作面，分层责任

用户真正面对的是 OPL App 或 OPL Workspace。两者都围绕项目、材料、任务、产物、审阅和下一步组织体验。Cloud 能力在需要时进入这条工作链，而不是成为另一套工作系统。

用户工作面	OPL App / OPL Workspace
AI 能力	OPL Gateway
资源连接与执行	OPL Fabric
组织政策与批准	OPL Console
来源与运行回执	OPL Ledger
专业质量判断	MAS / MAG / RCA 等领域 Agent
能力包生命周期	OPL Packages (由 OPL Framework 持有)

分层不是为了增加名称，而是为了避免一个系统拥有过多权力：

- Gateway 让模型接入、路由和用量有稳定入口。
- Fabric 连接计算、存储、环境、数据源与外部系统，并把一次任务绑定到获准资源。
- Console 管理组织、成员、额度、策略和批准，不承接日常专业工作，也不直接执行任务。
- Ledger 记录计划、批准、输入引用、环境、输出引用、审阅结果和继续入口，不保存第二份领域真相。
- 领域 Agent 保留专业策略、证据判断、质量结论与交付权威。
- OPL Packages 统一持有 Agent 与能力包的发现、安装、版本、lock、更新、回滚和修复；Cloud 各面只消费同一份 package refs，不再建立第二套注册表。

因此，一个用户可以只理解“我要完成什么、系统准备使用什么、结果在哪里”。模块边界留在系统背后，为这种简单体验提供可靠基础。

4 OPL Cloud 的五大设计原则

4.1 一、成果优先于基础设施

用户首先看到问题、材料、阶段、产物和下一步，而不是容器、队列、存储桶或内部服务名。只有当资源选择影响权限、成本或复查时，系统才展示必要信息。

这使复杂计算仍然表现为工作的一部分：用户批准的是“为这项分析使用实验室集群和指定环境”，收到的是图表、报告和审阅结果，而不是一串需要自行解释的作业编号。

4.2 二、本机与云端是同一条工作链

本机 OPL App 适合个人控制、敏感材料和日常编辑；在线 OPL Workspace 适合远程访问、团队协作和托管执行。两者不是彼此竞争的产品，而是同一工作模型的两种载体。

用户可以从本机开始，在需要团队访问或更强资源时接入 Cloud，再把结果带回原项目。环境改变，任务身份、材料引用、产物关系和继续入口不应丢失。

4.3 三、先计划，再授予资源权力

正式远端任务先形成资源计划：要访问哪些输入、使用什么环境与计算、写入什么位置、预计消耗什么、如何停止。个人或组织只对可理解的计划授权。

批准不是额外仪式，而是把危险动作和普通工作分开。低风险读取可以按既定策略直接进行；敏感数据出域、高成本计算、共享连接器和组织资源变更则在明确边界上停下来。

4.4 四、管理面不接管工作面

Console 服务管理员：组织、成员、权限、预算、额度、策略和批准。研究者、作者和设计者仍在 App 或 Workspace 中完成工作。

同样，能力包的安装、版本、lock、更新和 repair 归 OPL Packages；Console 只决定某个组织允许使用哪些已验证版本。Fabric 只为一次运行绑定所需资源。这样的单一 owner 设计，避免 Console、Fabric 和本机各自维护一份“当前版本”。

4.5 五、证据可以连接，不能越权裁决

Ledger 让计划、批准、环境、输入、输出和审阅彼此可追溯，但“有记录”不等于“结论正确”。科研结论由 MAS 等领域 Agent 和人类负责人判断；基金、视觉交付和书稿也各有自己的质量边界。

OPL Cloud 负责让判断有来路、结果能复查、工作可接力。它不会用运行成功、文件存在或回执齐全冒充专业 ready。

5 OPL Cloud 的能力版图

这不是一张功能菜单，而是用户在一条工作链中遇到的七个问题。每个问题只有一个主要责任方。

用户问题	主要责任方	用户获得什么
我在哪里继续工作？	OPL App / Workspace	连续的项目、任务、产物与审阅体验
AI 从哪里来、用了多少？	OPL Gateway	稳定模型入口、路由与用量信号
数据、工具和计算怎样接入？	OPL Fabric	可说明来源、环境与输出位置的资源绑定
团队允许谁使用什么？	OPL Console	组织策略、批准、额度与管理视图
Agent 和能力包现在是什么版本？	OPL Packages	已验证 manifest、安装状态、lock 与生命周期回执
这次运行发生了什么？	OPL Ledger	引用、回执、审阅结果与继续入口
结果在专业上是否成立？	领域 Agent / 人类 owner	领域质量判断、修订意见与交付决定

这里最重要的是 **不制造第二份 registry truth**。OPL Packages 的 package manifest、lock 和 lifecycle receipt 是能力包状态来源；Console 可以投影组织可用性，Fabric 可以读取执行所需绑定，Workspace 可以展示用户状态，Ledger 可以引用相关回执，但这些投影都不重新定义包的版本和安装事实。

同理，Ledger 不重新解释研究数据，Console 不替 Fabric 执行资源变更，Fabric 不替领域 Agent 判断结果，Workspace 不凭界面状态宣称服务已经 ready。用户看到的体验可以统一，背后的权威不能混在一起。

6 标准任务生命周期

下面是一条典型的实验室工作链。它比模块目录更能说明 OPL Cloud 为什么这样设计。

6.1 1. 从本机形成问题

研究者在 OPL App 中整理研究问题、分析计划和已有材料。患者级数据仍留在医院或机构存储，本机项目只保留允许使用的引用和上下文。

6.2 2. 选择已验证的专业能力

项目需要 MAS 及特定统计能力包。OPL Packages 读取经过验证的 manifest，确认已安装版本、依赖、lock 和当前健康状态。团队策略可以限制可用版本，但 Console 不另建包目录，也不修改 package truth。

6.3 3. 形成资源计划

任务发现本机算力不足，于是形成一份可读计划：在实验室 HPC 上运行、访问某个私有数据引用、使用指定 R/Python 环境、把脱敏结果写回项目输出区，并给出预算、停止方式和预期产物。

6.4 4. 在关键边界批准

Console 根据组织策略检查成员权限、HPC 配额、数据边界和预算。无需额外批准的动作按既定策略通过；涉及敏感数据或高成本资源时，由有权的人确认。

6.5 5. 在数据所在处执行

Fabric 绑定 HPC、私有存储和软件环境，提交并监控任务。数据不因“上云”而默认复制到中心系统；凭据和原始数据继续由资源 owner 持有。

6.6 6. 把结果带回工作台

计算完成后，图表、汇总和日志引用回到原项目。用户仍在熟悉的 App 或 Workspace 中查看结果、比较版本和决定下一步，不需要去基础设施页面拼装交付物。

6.7 7. 交给领域独立审阅

MAS 对方法、数字、图表与 claim 的对应关系进行专业审阅。运行成功不等于研究结论成立；发现问题时，修订意见回到同一工作链，必要时再次计划和执行。

6.8 8. 留下可继续的回执

Ledger 连接资源计划、批准、环境、输入引用、输出引用、审阅结果、负责人和继续入口。几个月后，团队可以从回执找到相关 owner source，而不是依赖一份复制出来、已经失去上下文的中央记录。

本机问题

- > 已验证能力包
- > 资源计划
- > 组织批准
- > 私有数据旁执行
- > 结果回到工作台
- > 领域独立审阅
- > 回执与下一轮继续

7 三类用户路径

同一设计可以按需要逐步展开，而不要求每个人一开始就采用完整 Cloud。

个人用户 可以始终以本机 App 为主，只在需要在线访问、Gateway 或远端计算时接入 Cloud。私有项目不会因为使用一项云能力就被迫迁移全部材料。

实验室与团队 可以共享批准过的模型、能力包、连接器、环境和计算资源。成员在各自工作面推进项目，管理员只处理组织策略，领域负责人继续对专业结果负责。

企业与机构 可以让内部数据库、私有存储、工具 API、HPC 或 GPU 保持原有 owner，同时通过 Fabric 接入标准执行，通过 Console 管理权限与预算，通过 Ledger 获得可复查引用。

三条路径共享的不是一套固定套餐，而是同一组设计判断：从成果出发、按需扩展、关键动作授权、权威各归其主。

8 系统模型

从用户视角看，OPL Cloud 只有三个时刻需要被感知：需要额外能力时、需要批准时、需要复查时。

时刻	用户看到	系统保证
需要额外能力	可理解的模型、环境或资源选择	Packages、Gateway 与 Fabric 各自提供当前 refs
需要批准	访问范围、成本、输出位置和停止方式	Console 执行组织策略，不接管执行
需要复查	产物、审阅结果、来源和继续入口	Ledger 连接 refs，领域 owner 保留结论权威

这种模型把复杂度放在系统内部，但不把它藏成不可解释的自动化。用户不必学习每个基础设施组件，却始终能知道系统准备做什么、谁批准、结果来自哪里、下一步由谁负责。

9 专业 workflow

科研只是其中一个例子。基金申请需要证据、版本、预算字段和模拟评审；汇报需要材料、故事线、图表、视觉审阅和导出；书稿需要章节、来源、风格与出版交接。这些工作共享云端资源和协作问题，却不能共享同一个质量结论。

OPL Cloud 因此只上收通用部分：工作面连续性、AI 接入、资源计划、权限批准、执行绑定、引用回执和继续入口。领域 Agent 保留专业方法、术语、质量标准、审阅行为和交付权威。

这种“通用基础设施 + 领域判断”的结构有两个好处：用户在不同专业任务中获得一致体验；产品又不会为了统一界面而稀释专业质量。

10 五类信任机制

10.1 范围可信

用户知道材料在哪里、任务准备访问什么、是否发生外部传输。敏感数据默认留在用户工作空间、机构存储或私有桶，Cloud 只保存必要的引用、策略和回执。

10.2 执行可信

远端任务说明计算位置、软件环境、输入引用、输出位置和停止方式。Fabric 的职责是忠实执行获准计划，并返回可检查的运行结果。

10.3 版本可信

OPL Packages 用 manifest、digest、lock 和 lifecycle receipt 表达安装、更新、回滚和 repair。App、Workspace、Console 与 Fabric 消费同一来源，不用多套 registry 猜测“当前版本”。

10.4 结果可信

Ledger 把运行证据交给领域审阅，但不把技术成功包装成专业成功。最终质量、发布、提交或交付决定来自对应领域 owner 和必要的人类 gate。

10.5 接力可信

重要结果带有负责人、输入输出 refs、审阅结果和继续入口。团队成员未来可以回到 owner source 重新理解和修订，而不是只能相信一份失去来源的快照。

11 本文边界

这份白皮书面向用户解释 OPL Cloud 的设计理念与目标产品边界。它回答“为什么这样设计会更好用”，不承担说明书、服务目录、发布公告或 readiness dashboard 的职责。

想确认的事实	应读取的权威来源
OPL Cloud 为什么存在、目标体验是什么	本白皮书
某个 Cloud 服务当前实现了什么	对应实现仓、接口合同与测试
某项能力此刻是否可用	运行读回、健康状态与 owner receipt
价格、套餐、安全与发布时间	正式产品页面、服务合同与公告
某份专业成果是否合格	MAS、MAG、RCA 等领域 owner 与人类负责人

当前 OPL Cloud 实现族正在按这些边界演进，但“进入目标架构”“合同已经存在”或“测试通过”都不等于全部服务已经交付。本文刻意保留这一差别，因为可信产品首先要诚实地区分方向、实现和运行事实。

12 结语

好的云端产品不应该让用户感到自己在操作一组云资源。它应该让用户继续思考问题、审阅结果和推进成果，同时需要在需要时获得更强 AI、在线工作空间、私有数据连接、远端计算和团队协作。

OPL Cloud 选择用连续工作面承接用户，用单一 owner 管理能力包，用 Fabric 连接资源，用 Console 克制地治理组织策略，用 Ledger 保存可追溯引用，再把专业判断交还领域 Agent 与人类负责人。

这套设计的价值不在于模块更多，而在于责任更清楚：工作可以跨环境继续，数据不必失去边界，管理不会淹没创作，证据不会冒充结论。用户因此能够相信，产品好用不是因为隐藏了复杂性，而是因为复杂性被放在了正确的位置。